

SERVLET + JDBC:-

Mode/Order.java:-

```
package model;

public class Order {

    private String name, item, address;

    private int quantity;

    private double total;

    public Order(String name, String item, int quantity, String address, double total) {

        this.name = name;

        this.item = item;

        this.quantity = quantity;

        this.address = address;

        this.total = total;

    }

    public String getName() { return name; }

    public String getItem() { return item; }

    public int getQuantity() { return quantity; }

    public String getAddress() { return address; }

    public double getTotal() { return total; }

}
```

Dao/orderdao.java:-

```
package dao;

import model.Order;

import java.sql.*;

public class OrderDAO {

    public void saveOrder(Order o) {

        try {

            Connection con = DBConnection.getConnection();
```

```

String q = "INSERT INTO orders(name,item,quantity,address,total) VALUES(?,?,?,?,?)";

PreparedStatement ps = con.prepareStatement(q);

ps.setString(1, o.getName());
ps.setString(2, o.getItem());
ps.setInt(3, o.getQuantity());
ps.setString(4, o.getAddress());
ps.setDouble(5, o.getTotal());

ps.executeUpdate();

} catch(Exception e) {
    e.printStackTrace();
}
}
}

```

OrderServlet.java:-

```

package controller;

import model.Order;
import dao.OrderDAO;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;
import java.util.*;

public class OrderServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        String name = req.getParameter("name");
    }
}

```

```
String item = req.getParameter("item");

int qty = Integer.parseInt(req.getParameter("quantity"));

String address = req.getParameter("address");


// Price logic

double price = 0;

if(item.equals("Pizza")) price = 200;

else if(item.equals("Burger")) price = 100;

else if(item.equals("Pasta")) price = 150;


double total = price * qty;


Order order = new Order(name, item, qty, address, total);


// Save to DB

new OrderDAO().saveOrder(order);


// SESSION (order history)

HttpSession session = req.getSession();

List<Order> list = (List<Order>) session.getAttribute("orders");


if(list == null) list = new ArrayList<>();

list.add(order);


session.setAttribute("orders", list);


// COOKIE (last item)

Cookie c = new Cookie("lastItem", item);

res.addCookie(c);


// Pass data to JSP

req.setAttribute("order", order);


RequestDispatcher rd = req.getRequestDispatcher("result.jsp");
```

```
        rd.forward(req, res);
    }
}
```

Html form:-

```
<h2>Food Order</h2>
```

```
<form action="OrderServlet" method="post">
```

```
    Name: <input type="text" name="name"><br>
```

Food:

```
<select name="item">
```

```
    <option>Pizza</option>
```

```
    <option>Burger</option>
```

```
    <option>Pasta</option>
```

```
</select><br>
```

```
    Quantity: <input type="number" name="quantity"><br>
```

```
    Address: <input type="text" name="address"><br>
```

```
    <input type="submit" value="Order">
```

```
</form>
```

Result.jsp:-

```
<%@ page import="model.Order,java.util.*" %>
```

```
<%
```

```
    Order o = (Order) request.getAttribute("order");
```

```
%>
```

```
<h2>Order Confirmation</h2>
```

```
Name: <%= o.getName() %> <br>
```

```
Item: <%= o.getItem() %> <br>
```

```
Quantity: <%= o.getQuantity() %> <br>
```

Total: ₹<%= o.getTotal() %>

<h3>Last Ordered Item (Cookie):</h3>

<%

Cookie[] cookies = request.getCookies();

if(cookies != null) {

for(Cookie c : cookies) {

if(c.getName().equals("lastItem")) {

out.println(c.getValue());

}

}

}

%>

JAVAFX WITH JDBC:-

Mainapp.java:-

import javafx.application.Application;

import javafx.scene.Scene;

import javafx.scene.control.*;

import javafx.scene.layout.VBox;

import javafx.stage.Stage;

import java.sql.*;

public class MainApp extends Application {

// DB connection method

public Connection getConnection() {

try {

Class.forName("com.mysql.cj.jdbc.Driver");

return DriverManager.getConnection(

"jdbc:mysql://localhost:3306/examdb",

"root",

```

        "YOUR_PASSWORD"
    );
} catch (Exception e) {
    e.printStackTrace();
    return null;
}
}

```

```

@Override

```

```

public void start(Stage stage) {

```

```

    TextField name = new TextField();
    name.setPromptText("Name");

```

```

    TextField email = new TextField();
    email.setPromptText("Email");

```

```

    TextField course = new TextField();
    course.setPromptText("Course");

```

```

    Button addBtn = new Button("Add Student");
    Button loadBtn = new Button("Load Students");

```

```

    TextArea output = new TextArea();

```

```

// ADD STUDENT

```

```

addBtn.setOnAction(e -> {

```

```

    try {

```

```

        Connection con = getConnection();

```

```

        String q = "INSERT INTO students(name,email,course) VALUES(?,?,?)";

```

```

        PreparedStatement ps = con.prepareStatement(q);

```

```

        ps.setString(1, name.getText());

```

```

        ps.setString(2, email.getText());

        ps.setString(3, course.getText());


        ps.executeUpdate();


        output.setText("Student Added Successfully!");


    } catch (Exception ex) {
        ex.printStackTrace();
    }
});


// LOAD STUDENTS
loadBtn.setOnAction(e -> {
    try {
        Connection con = getConnection();

        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM students");

        String data = "";

        while(rs.next()){
            data += rs.getInt("id") + " | "
                + rs.getString("name") + " | "
                + rs.getString("email") + " | "
                + rs.getString("course") + "\n";
        }

        output.setText(data);

    } catch (Exception ex) {
        ex.printStackTrace();
    }
}

```

```
});
```

```
VBox root = new VBox(10, name, email, course, addBtn, loadBtn, output);
```

```
stage.setScene(new Scene(root, 400, 400));
```

```
stage.setTitle("Student Registration System");
```

```
stage.show();
```

```
}
```

```
public static void main(String[] args) {
```

```
    launch();
```

```
}
```

```
}
```

MULTITHREADING + EXCEPTION:-

```
import java.util.*;
```

```
// Custom Exception
```

```
class InsufficientBalanceException extends Exception {
```

```
    InsufficientBalanceException(String msg) {
```

```
        super(msg);
```

```
    }
```

```
}
```

```
// User Class
```

```
class User {
```

```
    String name;
```

```
    int age;
```

```
    User(String name, int age) {
```

```
        this.name = name;
```

```
        setAge(age);
```

```
    }
```

```
    void setAge(int age) {
```



```

        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or above");
        }

        this.age = age;
    }
}

// Bank Account Class

class BankAccount {

    double balance;

    BankAccount(double balance) {

        this.balance = balance;
    }

    // Withdraw (Synchronized)

    synchronized void withdraw(double amount) throws InsufficientBalanceException {

        if (amount > balance) {

            throw new InsufficientBalanceException("Insufficient Balance!");

        }

        balance -= amount;

        System.out.println(Thread.currentThread().getName() +

            " withdrew: " + amount);

        System.out.println("Remaining Balance: " + balance);

    }

    // Deposit (Synchronized)

    synchronized void deposit(double amount) {

        balance += amount;

        System.out.println(Thread.currentThread().getName() +

            " deposited: " + amount);

        System.out.println("Updated Balance: " + balance);

    }
}

```

```
}
```

```
// Withdraw Thread
```

```
class WithdrawThread extends Thread {
```

```
    BankAccount acc;
```

```
    double amount;
```

```
    WithdrawThread(BankAccount acc, double amount) {
```

```
        this.acc = acc;
```

```
        this.amount = amount;
```

```
    }
```

```
    public void run() {
```

```
        try {
```

```
            acc.withdraw(amount);
```

```
        } catch (Exception e) {
```

```
            System.out.println(e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

```
// Deposit Runnable
```

```
class DepositRunnable implements Runnable {
```

```
    BankAccount acc;
```

```
    double amount;
```

```
    DepositRunnable(BankAccount acc, double amount) {
```

```
        this.acc = acc;
```

```
        this.amount = amount;
```

```
    }
```

```
    public void run() {
```

```
        acc.deposit(amount);
```

```
    }
```

```
}
```

```
// Main Class
```

```
public class BankSystem {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            System.out.println("Enter User Name:");
```

```
            String name = sc.nextLine();
```

```
            System.out.println("Enter Age:");
```

```
            int age = sc.nextInt();
```

```
            User u = new User(name, age);
```

```
            System.out.println("Enter Initial Balance:");
```

```
            double bal = sc.nextDouble();
```

```
            BankAccount acc = new BankAccount(bal);
```

```
            System.out.println("\n1. Withdraw");
```

```
            System.out.println("2. Deposit");
```

```
            System.out.println("3. Test Exceptions");
```

```
            System.out.println("Enter choice:");
```

```
            int ch = sc.nextInt();
```

```
            switch (ch) {
```

```
                case 1:
```

```
                    System.out.println("Enter amount to withdraw:");
```

```
                    double w = sc.nextDouble();
```

```
WithdrawThread t1 = new WithdrawThread(acc, w);  
WithdrawThread t2 = new WithdrawThread(acc, w / 2);
```

```
t1.start();  
t2.start();  
  
break;
```

case 2:

```
System.out.println("Enter amount to deposit:");  
  
double d = sc.nextDouble();  
  
Thread t3 = new Thread(new DepositRunnable(acc, d));  
t3.start();  
  
break;
```

case 3:

```
try {  
    int x = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("ArithmeticException handled");  
}  
  
try {  
    int n = Integer.parseInt("abc");  
} catch (NumberFormatException e) {  
    System.out.println("NumberFormatException handled");  
}  
  
try {  
    String s = null;  
    System.out.println(s.length());  
} catch (NullPointerException e) {  
    System.out.println("NullPointerException handled");  
}
```

```

    }

    try {
        int arr[] = new int[2];

        arr[5] = 10;
    } catch (ArrayIndexOutOfBoundsException e) {

        System.out.println("ArrayIndexOutOfBoundsException handled");
    }

    try {
        u.setAge(15);
    } catch (IllegalArgumentException e) {

        System.out.println(e.getMessage());
    }

    break;

    default:

        System.out.println("Invalid Choice");
    }

} catch (Exception e) {

    System.out.println("Invalid Input");
}

}

sc.close();
}
}

```

// TourismPlanner.java

```
import javafx.application.Application;

import javafx.scene.Scene;

import javafx.scene.control.*;

import javafx.scene.layout.GridPane;

import javafx.stage.Stage;

import java.sql.*;

public class TourismPlanner extends Application {

    TextField placeField = new TextField();

    TextArea resultArea = new TextArea();

    public static void main(String[] args) {

        launch(args);

    }

    public void start(Stage stage) {

        GridPane grid = new GridPane();

        Button searchBtn = new Button("Search");

        grid.add(new Label("Place:"), 0, 0);

        grid.add(placeField, 1, 0);

        grid.add(searchBtn, 1, 1);

        grid.add(resultArea, 1, 2);

        searchBtn.setOnAction(e -> searchPlace());

        stage.setScene(new Scene(grid, 400, 300));

        stage.setTitle("Tourism Planner");

        stage.show();

    }

    void searchPlace() {
```

```

try {

    Connection con = DriverManager.getConnection(

        "jdbc:mysql://localhost:3306/tourism", "root", "password");

    PreparedStatement ps = con.prepareStatement(

        "SELECT * FROM tourist_places WHERE name=?");

    ps.setString(1, placeField.getText());

    ResultSet rs = ps.executeQuery();

    if (rs.next()) {

        resultArea.setText(

            "Place: " + rs.getString("name") +

            "\nCost: " + rs.getDouble("stay_cost") +

            "\nTravel Cost: " + rs.getDouble("travel_cost"));

    } else {

        resultArea.setText("No data found");

    }

} catch (Exception e) {

    e.printStackTrace();

}

}

```

// InventorySystem.java

```

import javafx.application.Application;

import javafx.scene.Scene;

import javafx.scene.control.*;

import javafx.scene.layout.GridPane;

import javafx.stage.Stage;

import java.sql.*;

```

```

public class InventorySystem extends Application {

```

```
TextField id = new TextField();

TextField name = new TextField();

TextField qty = new TextField();

TextField price = new TextField();


public static void main(String[] args) {

    launch(args);

}


public void start(Stage stage) {

    GridPane grid = new GridPane();

    Button addBtn = new Button("Add Item");

    grid.add(new Label("ID"), 0, 0);
    grid.add(id, 1, 0);
    grid.add(new Label("Name"), 0, 1);
    grid.add(name, 1, 1);
    grid.add(new Label("Qty"), 0, 2);
    grid.add(qty, 1, 2);
    grid.add(new Label("Price"), 0, 3);
    grid.add(price, 1, 3);
    grid.add(addBtn, 1, 4);

    addBtn.setOnAction(e -> addItem());

    stage.setScene(new Scene(grid, 300, 250));
    stage.show();

}


void addItem() {

    try {

        Connection con = DriverManager.getConnection(
```



```

        "jdbc:mysql://localhost:3306/shop", "root", "password");

PreparedStatement ps = con.prepareStatement(
    "INSERT INTO inventory VALUES (?, ?, ?, ?)");

ps.setInt(1, Integer.parseInt(id.getText()));
ps.setString(2, name.getText());
ps.setInt(3, Integer.parseInt(qty.getText()));
ps.setDouble(4, Double.parseDouble(price.getText()));

ps.executeUpdate();

System.out.println("Item Added");

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Shopping mall

```

package tourismplanner; // you can keep same package

// ----- IMPORTS -----

import javafx.application.Application;

import javafx.scene.Scene;

import javafx.scene.control.*;

import javafx.scene.layout.VBox;

import javafx.stage.Stage;

```

```
import java.sql.*;

public class MallApp extends Application {

    // ----- DATABASE CONNECTION -----

    public Connection connectDB() {

        try {

            return DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/mall_db",

                "root",

                "password" // change if needed

            );

        } catch (Exception e) {

            e.printStackTrace();

            return null;

        }

    }

    @Override

    public void start(Stage stage) {

        // ===== INVENTORY INPUT =====

        TextField itemId = new TextField();

        itemId.setPromptText("Item ID");

        TextField itemName = new TextField();

        itemName.setPromptText("Item Name");

        TextField qty = new TextField();

        qty.setPromptText("Quantity");

        TextField price = new TextField();
```

```
price.setPromptText("Price per item");
```

```
Button addItemBtn = new Button("Add Item");
```

```
// ===== SALES INPUT =====
```

```
TextField mobile = new TextField();
```

```
mobile.setPromptText("Customer Mobile");
```

```
TextField saleItemId = new TextField();
```

```
saleItemId.setPromptText("Item ID");
```

```
TextField saleQty = new TextField();
```

```
saleQty.setPromptText("Quantity");
```

```
Button saleBtn = new Button("Make Sale");
```

```
// ===== OUTPUT AREA =====
```

```
TextArea output = new TextArea();
```

```
// ===== ADD ITEM FUNCTION =====
```

```
addItemBtn.setOnAction(e -> {
```

```
    try {
```

```
        Connection con = connectDB();
```

```
        String query = "INSERT INTO items VALUES (?, ?, ?, ?)";
```

```
        PreparedStatement ps = con.prepareStatement(query);
```

```
        ps.setInt(1, Integer.parseInt(itemId.getText()));
```

```
        ps.setString(2, itemName.getText());
```

```
        ps.setInt(3, Integer.parseInt(qty.getText()));
```

```

ps.setDouble(4, Double.parseDouble(price.getText()));

ps.executeUpdate();

output.setText("✅ Item Added Successfully");

} catch (Exception ex) {
    output.setText("❌ Error: " + ex.getMessage());
}
});

// ===== SALES FUNCTION =====

saleBtn.setOnAction(e -> {
    try {
        Connection con = connectDB();

        int id = Integer.parseInt(saleItemId.getText());
        int quantity = Integer.parseInt(saleQty.getText());

        // 1. Get item price and current stock
        String selectQuery = "SELECT * FROM items WHERE item_id=?";
        PreparedStatement ps1 = con.prepareStatement(selectQuery);
        ps1.setInt(1, id);

        ResultSet rs = ps1.executeQuery();

        if (rs.next()) {

            int currentQty = rs.getInt("qty");
            double pricePerItem = rs.getDouble("price");

            // Check stock availability
            if (currentQty >= quantity) {

```

```

double totalCost = quantity * pricePerItem;

// 2. Insert into sales table
String insertSale = "INSERT INTO sales(mobile, item_id, qty, cost) VALUES (?, ?, ?, ?)";
PreparedStatement ps2 = con.prepareStatement(insertSale);

ps2.setString(1, mobile.getText());
ps2.setInt(2, id);
ps2.setInt(3, quantity);
ps2.setDouble(4, totalCost);

ps2.executeUpdate();

// 3. Update stock (reduce quantity)
String updateStock = "UPDATE items SET qty = qty - ? WHERE item_id=?";
PreparedStatement ps3 = con.prepareStatement(updateStock);

ps3.setInt(1, quantity);
ps3.setInt(2, id);

ps3.executeUpdate();

output.setText("✅ Sale Done! Total Cost = ₹" + totalCost);

} else {
    output.setText("❌ Not enough stock!");
}

} else {
    output.setText("❌ Item not found!");
}
}

```

```

    } catch (Exception ex) {

        output.setText(" ❌ Error: " + ex.getMessage());

    }

});

// ===== LAYOUT =====

VBox layout = new VBox(10);

layout.getChildren().addAll(

    new Label("=== Add Inventory ==="),

    itemId, itemName, qty, price, addItemBtn,

    new Label("=== Sales Entry ==="),

    mobile, saleItemId, saleQty, saleBtn,

    output

);

Scene scene = new Scene(layout, 400, 600);

stage.setTitle("Shopping Mall System");
stage.setScene(scene);
stage.show();

}

public static void main(String[] args) {

    launch();

}

}

```

Servlet connection

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

import java.sql.*;

import java.time.*;

public class VoterServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        // ----- FORM DATA GET -----

        String name = req.getParameter("name");

        String aadhaar = req.getParameter("aadhaar");

        String dob = req.getParameter("dob");

        String citizenship = req.getParameter("citizenship");

        String mobile = req.getParameter("mobile");

        String address = req.getParameter("address");

        String city = req.getParameter("city");

        String state = req.getParameter("state");

        String pincode = req.getParameter("pincode");

        PrintWriter out = res.getWriter();

        try {

            Connection con = DBConnection.getCon();

            // ----- AGE CHECK -----

            LocalDate birthDate = LocalDate.parse(dob);

            int age = Period.between(birthDate, LocalDate.now()).getYears();

            if (age < 18) {
```

```

        out.println(" ❌ Age must be 18+");

        return;
    }

    // ----- CITIZENSHIP CHECK -----

    if (!citizenship.equalsIgnoreCase("India")) {

        out.println(" ❌ Only Indian citizens allowed");

        return;
    }

    // ----- DUPLICATE CHECK -----

    PreparedStatement ps1 = con.prepareStatement(

        "SELECT * FROM voters WHERE aadhaar=?"

    );

    ps1.setString(1, aadhaar);

    ResultSet rs = ps1.executeQuery();

    if (rs.next()) {

        out.println(" ❌ Aadhaar already exists");

        return;
    }

    // ----- INSERT DATA -----

    PreparedStatement ps2 = con.prepareStatement(

        "INSERT INTO voters(name, aadhaar, dob, citizenship, mobile, address, city, state, pincode) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"

    );

    ps2.setString(1, name);

    ps2.setString(2, aadhaar);

    ps2.setString(3, dob);

    ps2.setString(4, citizenship);

    ps2.setString(5, mobile);

```



```

        ps2.setString(6, address);

        ps2.setString(7, city);

        ps2.setString(8, state);

        ps2.setString(9, pincode);


        ps2.executeUpdate();


        out.println("✅ Voter Registered Successfully");


    } catch (Exception e) {

        out.println("Error: " + e.getMessage());

    }

}

}

package model;

import java.sql.Date;

// Model class → represents Voter entity
public class Voter {

    private String name, aadhaar, citizenship, mobile, address, city, state, pincode;

    private Date dob;


    // Getter & Setter methods (used to store and retrieve data)


    public String getName() { return name; }

    public void setName(String name) { this.name = name; }


    public String getAadhaar() { return aadhaar; }

    public void setAadhaar(String aadhaar) { this.aadhaar = aadhaar; }

```

```
public Date getDob() { return dob; }

public void setDob(Date dob) { this.dob = dob; }


public String getCitizenship() { return citizenship; }

public void setCitizenship(String citizenship) { this.citizenship = citizenship; }


public String getMobile() { return mobile; }

public void setMobile(String mobile) { this.mobile = mobile; }


public String getAddress() { return address; }

public void setAddress(String address) { this.address = address; }


public String getCity() { return city; }

public void setCity(String city) { this.city = city; }


public String getState() { return state; }

public void setState(String state) { this.state = state; }


public String getPincode() { return pincode; }

public void setPincode(String pincode) { this.pincode = pincode; }

}
```

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

import java.sql.*;

import java.time.*;
```

```
public class VoterServlet extends HttpServlet {
```

```
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {

    // ----- FORM DATA GET -----

    String name = req.getParameter("name");

    String aadhaar = req.getParameter("aadhaar");

    String dob = req.getParameter("dob");

    String citizenship = req.getParameter("citizenship");

    String mobile = req.getParameter("mobile");

    String address = req.getParameter("address");

    String city = req.getParameter("city");

    String state = req.getParameter("state");

    String pincode = req.getParameter("pincode");

    PrintWriter out = res.getWriter();

    try {

        Connection con = DBConnection.getCon();

        // ----- AGE CHECK -----

        LocalDate birthDate = LocalDate.parse(dob);

        int age = Period.between(birthDate, LocalDate.now()).getYears();

        if (age < 18) {

            out.println(" ❌ Age must be 18+");

            return;

        }

        // ----- CITIZENSHIP CHECK -----

        if (!citizenship.equalsIgnoreCase("India")) {

            out.println(" ❌ Only Indian citizens allowed");

            return;

        }

    }
```

```

// ----- DUPLICATE CHECK -----

PreparedStatement ps1 = con.prepareStatement(

    "SELECT * FROM voters WHERE aadhaar=?"

);

ps1.setString(1, aadhaar);

ResultSet rs = ps1.executeQuery();

if (rs.next()) {

    out.println(" ❌ Aadhaar already exists");

    return;

}

// ----- INSERT DATA -----

PreparedStatement ps2 = con.prepareStatement(

    "INSERT INTO voters(name, aadhaar, dob, citizenship, mobile, address, city, state, pincode) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"

);

ps2.setString(1, name);

ps2.setString(2, aadhaar);

ps2.setString(3, dob);

ps2.setString(4, citizenship);

ps2.setString(5, mobile);

ps2.setString(6, address);

ps2.setString(7, city);

ps2.setString(8, state);

ps2.setString(9, pincode);

ps2.executeUpdate();

out.println(" ✅ Voter Registered Successfully");

} catch (Exception e) {

```

```
        out.println("Error: " + e.getMessage());
    }
}
}
```

Index.jsp

```
<form action="VoterServlet" method="post">

    Name: <input type="text" name="name"><br>
    Aadhaar: <input type="text" name="aadhaar"><br>
    DOB: <input type="date" name="dob"><br>
    Citizenship: <input type="text" name="citizenship"><br>
    Mobile: <input type="text" name="mobile"><br>
    Address: <input type="text" name="address"><br>
    City: <input type="text" name="city"><br>
    State: <input type="text" name="state"><br>
    Pincode: <input type="text" name="pincode"><br>

    <input type="submit" value="Register">

</form>
```

Search.jsp

```
<form action="SearchServlet" method="get">

    Enter State: <input type="text" name="state">

    <input type="submit" value="Search">

</form>
```

State search servlet

```
<form action="VoterServlet" method="post">

    Name: <input type="text" name="name"><br>

    Aadhaar: <input type="text" name="aadhaar"><br>

    DOB: <input type="date" name="dob"><br>

    Citizenship: <input type="text" name="citizenship"><br>

    Mobile: <input type="text" name="mobile"><br>

    Address: <input type="text" name="address"><br>

    City: <input type="text" name="city"><br>

    State: <input type="text" name="state"><br>

    Pincode: <input type="text" name="pincode"><br>

    <input type="submit" value="Register">

</form>
```

Web.xml

```
<web-app>

    <servlet>

        <servlet-name>VoterServlet</servlet-name>

        <servlet-class>com.voter.VoterServlet</servlet-class>

    </servlet>

    <servlet-mapping>

        <servlet-name>VoterServlet</servlet-name>

        <url-pattern>/VoterServlet</url-pattern>

    </servlet-mapping>

    <servlet>

        <servlet-name>SearchServlet</servlet-name>

        <servlet-class>com.voter.SearchServlet</servlet-class>

    </servlet>
```

```
<servlet-mapping>
    <servlet-name>SearchServlet</servlet-name>
    <url-pattern>/SearchServlet</url-pattern>
</servlet-mapping>
```

```
</web-app>
```

Jomato

```
<!-- Order form -->
```

```
<form action="OrderServlet" method="post">
```

Dish Type:

```
<select name="type">
    <option>Veg</option>
    <option>Non-Veg</option>
</select><br>
```

Dish Name: <input type="text" name="dish">

Quantity: <input type="number" name="qty">

Address: <input type="text" name="address">

Mobile: <input type="text" name="mobile">

Contact Person: <input type="text" name="person">


```
<input type="submit" value="Order">
```

```
</form>
```

```
package com.jomoto;
```

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
public class OrderServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest req, HttpServletResponse res)  
        throws IOException {
```

```
        // ----- FORM DATA -----
```

```
        String type = req.getParameter("type");
```

```
        String dish = req.getParameter("dish");
```

```
        int qty = Integer.parseInt(req.getParameter("qty"));
```

```
        String address = req.getParameter("address");
```

```
        String mobile = req.getParameter("mobile");
```

```
        String person = req.getParameter("person");
```

```
        PrintWriter out = res.getWriter();
```

```
        // ----- PRICE FIX (simple assumption) -----
```

```
        int pricePerItem = 100; // har dish ka price 100 maan liya
```

```
        double total = qty * pricePerItem;
```

```
        boolean isOldUser = false;
```

```
        // ----- COOKIE CHECK -----
```

```
        Cookie[] cookies = req.getCookies();
```

```
        if (cookies != null) {
```

```
            for (Cookie c : cookies) {
```



```
// agar cookie mil gaya matlab user pehle aa chuka hai

if (c.getName().equals("visited")) {

    isOldUser = true;

}

}

}
```

```
// ----- DISCOUNT LOGIC -----
```

```
if (isOldUser) {

    total = total - (total * 0.05); // 5% discount

}
```

```
// ----- NEW COOKIE SET KARNA -----
```

```
Cookie cookie = new Cookie("visited", "yes");
```

```
// cookie 1 day ke liye store hogi
```

```
cookie.setMaxAge(60 * 60 * 24);
```

```
res.addCookie(cookie);
```

```
// ----- OUTPUT -----
```

```
out.println("<h2>Order Details</h2>");

out.println("Dish: " + dish + "<br>");

out.println("Type: " + type + "<br>");

out.println("Quantity: " + qty + "<br>");

out.println("Address: " + address + "<br>");

out.println("Mobile: " + mobile + "<br>");

out.println("Person: " + person + "<br>");
```

```
if (isOldUser) {

    out.println("<h3> 🎉 5% Discount Applied!</h3>");

} else {

    out.println("<h3>First Order (No Discount)</h3>");

}
```

```
}

    out.println("<h2>Total Bill: ₹" + total + "</h2>");
}
}
```

```
<web-app>

    <servlet>

        <servlet-name>OrderServlet</servlet-name>

        <servlet-class>com.jomoto.OrderServlet</servlet-class>

    </servlet>

    <servlet-mapping>

        <servlet-name>OrderServlet</servlet-name>

        <url-pattern>/OrderServlet</url-pattern>

    </servlet-mapping>

</web-app>
```